

Add Patient Module

Playwright E2E Automation — Study Notes

Metabolic Staff Portal | June 2026

1. What We Built

The Add Patient module automates the process of creating a new patient in the Metabolic Staff Portal and verifying them. Two files make up the entire module:

File	Role
pages/AddPatientPage.js	Page Object Model — all locators and actions for the Add Patient form
tests/e2e/add-patient.spec.js	Test file — 9 tests covering the full Add Patient flow

2. The Page Object Model (POM) Pattern

The core idea: Never write locators (CSS selectors, XPaths, role finders) directly inside test files. Instead, put them in a separate "Page" class. Tests only call methods — they never know about selectors.

```
// ✅ Good – test reads like a sentence
await addPage.fillForm(patient);
await addPage.submit();

// ❌ Bad – locator logic mixed into test
await page.getByRole('textbox', { name: 'First Name' }).fill('Auto');
await page.getByRole('dialog').getByRole('button', { name: 'Add Patient' }).click();
```

Why this matters

If the app changes a button's ID tomorrow, you fix it in ONE place (the page object), not in every test that uses it.

3. AddPatientPage.js — Key Concepts

3a. Constructor — Defining Locators

The constructor runs when you do `new AddPatientPage(page)`. It sets up every locator as a class property so any method or test can reuse them.

```
constructor(page) {
  this.page = page;

  // Always prefer getByRole - it works like a real user would
  this.firstNameInput = page.getByRole('textbox', { name: 'First Name' });
  this.modal          = page.getByRole('dialog');

  // Scoped to the dialog - avoids matching the "Add Patient" button in the header
  this.submitBtn = page.getByRole('dialog').getByRole('button', { name: 'Add Patient' });
}
```

Why getByRole?

It's the most stable selector — it finds elements the same way a user (or screen reader) would. It doesn't break when CSS classes change.

3b. React-Select Dropdowns

The form uses a library called **React-Select** for dropdowns (Insurance Provider, Language, Nationality, Diagnosis, Teams). These are NOT native `<select>` elements — they are custom-built React components.

```
// ❌ This doesn't work - not a native select element
await page.selectOption('AXA Insurance');

// ✅ Correct approach - type into hidden input, pick from options list
async selectLanguage(searchText) {
  await this.languageInput.fill(searchText); // type to search
  await this.languageInput.press('Enter');   // confirm selection
}
```

Each React-Select has a hidden `<input>` with a generated ID like `react-select-3-input`. You type into it, matching options appear in a dropdown list, then you press Enter or click an option.

3c. The Date of Birth Field — `pressSequentially` + `Tab`

```
async fillDateOfBirth(dateString) {  
  await this.dobInput.click();  
  await this.dobInput.pressSequentially(dateString);  
  await this.dobInput.press('Tab');    // ← NOT Escape  
}
```

Decision	Reason
<code>pressSequentially</code> not <code>fill()</code>	The date field is a masked input (DD-MM-YYYY). It needs keystrokes one at a time to move through each segment: day → month → year.
<code>Tab</code> not <code>Escape</code>	The form uses a Bootstrap Offcanvas (slide-in panel). Bootstrap listens for <code>Escape</code> globally — pressing it closes the ENTIRE modal. <code>Tab</code> just moves focus safely to the next field.

3d. The React Re-render Problem — Fill Email LAST

This was the hardest bug in the entire module.

The form would appear to fill correctly, but on submission the email field was always empty.

What happened: Each `React-Select` interaction caused React to re-render the form. When React re-renders, it resets any input it doesn't currently "own" — meaning our previously typed email value was wiped from the DOM.

```

async fillForm(...) {
  // STEP 1 – Fill stable text fields (not affected by re-renders)
  await this.firstNameInput.fill(firstName);
  await this.lastNameInput.fill(lastName);
  await this.fillDateOfBirth(dob);
  await this.mrnInput.fill(mrn);

  // STEP 2 – React-selects (each one triggers a re-render)
  await this.selectInsuranceProvider(insuranceProvider);
  await this.selectLanguage(language);
  await this.selectNationality(nationality);
  await this.selectDiagnosis(diagnosis);
  await this.selectPrimaryTeam(primaryTeam);
  await this.selectSecondaryTeam(... secondaryTeam);

  // STEP 3 – Email and phone filled LAST
  // Re-renders from Step 2 would wipe these if filled earlier
  await this.phoneCountrySelect.selectOption(phoneCountry);
  await this.emailInput.click();
  await this.emailInput.fill(email);
  await this.phoneInput.fill(phone);
}

```

Rule to remember

In React forms, always fill fields that trigger re-renders (dropdowns) before fields that get wiped by those re-renders (plain text inputs like email).

4. Test File — Key Concepts

4a. Serial Mode

```
test.describe.configure({ mode: 'serial' });
```

Tests inside this describe block run **one after another**, never in parallel. This is required because:

- Test 7 (happy path) **creates** the patient
- Test 8 (verify patient) **depends on** that patient existing

Without serial mode, Test 8 could start before Test 7 finishes — and fail because the patient doesn't exist yet.

4b. Unique Test Data Per Run

```
const uid      = Date.now();           // e.g. 1750000123456 (13 digits)
const shortId  = uid.toString().slice(-6); // e.g. "123456" (last 6 digits)

const patient = {
  lastName: `AT${shortId}`,           // AT123456
  email:    `at${shortId}@getnada.com`, // at123456@getnada.com
  mrn:      `AT${shortId}`,           // AT123456
  phone:    '3' + Math.floor(Math.random() * 1_000_000_000)
    .toString().padStart(9, '0'),
};
```

Field	Why it must be unique	Strategy used
Email	App rejects duplicate emails	Last 6 digits of timestamp
MRN	App rejects duplicate MRNs	Last 6 digits of timestamp
Phone	App rejects duplicate phones	Random 9-digit number starting with 3

Why last 6 digits, not all 13?

The MRN field has a character limit. Using all 13 digits of `Date.now()` would get truncated to 10 characters — and the first digits of a timestamp repeat across days, causing collisions. The last 6 digits change every millisecond and fit within the limit.

4c. beforeEach — Shared Setup

```
test.beforeEach(async ({ page }) => {
  listingsPage = new PatientListingsPage(page);
  addPage       = new AddPatientPage(page);
  await listingsPage.goto();
  // Wait for a table row - NOT networkidle (see section 4d)
  await page.getByRole('row').nth(1).waitFor({ state: 'visible', timeout: 15000 });
});
```

`beforeEach` runs automatically before every single test. It navigates to the patient listings and waits for the table to load.

4d. Why We Never Use `waitForLoadState('networkidle')`

This app has a live chat widget

that holds a permanent open WebSocket connection. Playwright's `networkidle` waits until ALL network activity stops — but it never does. Every test that used `networkidle` would hang until the 30-second timeout expired.

Instead of	Use
<code>waitForLoadState('networkidle')</code> — page load	<code>waitFor({ state: 'visible' })</code> on a specific element
<code>waitForLoadState('networkidle')</code> — after navigation	<code>waitForLoadState('domcontentloaded')</code>
<code>waitForLoadState('networkidle')</code> — after search	<code>waitForTimeout(800)</code> for debounced inputs

4e. The `Promise.all` Pattern

When an action (like filling a search box) triggers an API call, you need to wait for the response. But there's a timing trap:

```
// ❌ WRONG - Race condition
// The response might arrive BEFORE the listener is registered
await this.searchInput.fill(name);
await this.page.waitForResponse(...); // too late - already missed it

// ✅ CORRECT - Both start at the same instant
// Listener is registered before the browser processes the fill event
await Promise.all([
  this.page.waitForResponse(...), // registers listener
  this.searchInput.fill(name),    // triggers the action
]);
```

How Promise.all works here

JavaScript is single-threaded. `Promise.all` initialises both promises synchronously before either runs. So the response listener is always registered before the fill event is dispatched to the browser.

4f. Happy Path Assertion — Why Not `toHaveURL`?

```
// ❌ WRONG – URL was already /patient before the modal opened
await expect(page).toHaveURL(/\/patient/);

// ✅ CORRECT – Modal disappearing proves the form submitted successfully
await expect(addPage.modal).not.toBeVisible({ timeout: 30000 });
```

The page URL was `/patient` before and after adding a patient. Checking URL told us nothing. The modal only closes when the app receives a successful API response — so asserting it's gone is the real proof of success.

5. Verify Patient — Locator Lessons

Problem 1: Wrong button found for three-dots menu

The profile header has a three-dots action button. It uses a Bootstrap Dropdown which gives it `id="dropdown-basic"`. But the Primary Team section's avatar button ALSO uses a Bootstrap Dropdown with the same ID. After verification, both are on the page simultaneously.

```
// ❌ Fails – strict mode: 2 elements found
await page.locator('#dropdown-basic').click();

// ✅ Fix – always target the first one (header action button loads first)
await page.locator('#dropdown-basic').first().waitFor({ state: 'visible', timeout: 1000 });
await page.locator('#dropdown-basic').first().click();
```

Problem 2: `getByText('Verify')` matching wrong elements

```
// ❌ Can match any text on the page containing "Verify"
await page.getByText('Verify', { exact: true }).click();

// ✅ Scoped to dropdown menu items specifically
// data-rr-ui-dropdown-item is a React Bootstrap internal attribute – stable
await page.locator('[data-rr-ui-dropdown-item]')
    .filter({ hasText: 'Verify' })
    .click();
```

Problem 3: Asserting VERIFIED on the wrong page

After verifying, the test was looking for a "VERIFIED" badge on the patient profile. But that badge only exists in the listings table.

```
// ❌ Wrong – profile page doesn't have a VERIFIED badge
await expect(page.getByText('VERIFIED')).toBeVisible();

// ✅ Correct – go back to listings, search, check the badge there
await listingsPage.goto();
await listingsPage.searchPatient(patient.lastName);
await expect(page.getByText('VERIFIED').first()).toBeVisible({ timeout: 10000 });
```


6. All Tests at a Glance

#	Test Name	What It Checks
1	Open modal	Clicking "Add Patient" shows the offcanvas form
2	Close modal	Clicking "Cancel" hides the form
3	Form fields visible	All required inputs are present in the modal
4	Empty form validation	Form doesn't submit with no data — modal stays open
5	Missing First Name	Validation fires, modal stays open
6	Missing Last Name	Validation fires, modal stays open
7	Happy path — Add Patient	Full form fills and submits, patient appears in search results
8	Verify patient	Three-dots → Verify → Yes → VERIFIED badge visible in listings
9	MRN Lookup SKIPPED	Details to be provided separately
10	Invalid MRN Error SKIPPED	Details to be provided separately

7. Key Takeaways

1. **POM keeps tests clean** — locators live in one place, tests read like plain English sentences.
2. **React re-renders wipe inputs** — always fill email and phone AFTER all dropdowns.
3. **networkidle is broken here** — the live chat widget holds a permanent network connection. Use element-based waits instead.
4. **Promise.all prevents race conditions** — the response listener must be registered before the action fires.
5. **pressSequentially for masked inputs** — `fill()` doesn't work on character-by-character date fields.
6. **Tab not Escape** — Bootstrap Offcanvas hijacks the Escape key globally and closes the modal.
7. **Unique data per run** — use `Date.now().slice(-6)` for MRN/email and `Math.random()` for phone to avoid app-level uniqueness rejections.
8. **Assert what actually changes** — check the modal closes (not the URL); check the VERIFIED badge in listings (not on the profile page).

